MEMRES

Memory-Augmented Agentic Python Dependency Resolver

Chi-Nguyen Tran · Sy-Duy-Minh Dao · Trung-Kiet Huynh · Phu-Hoa Pham · Lam-Phu-Quy Nguyen

University of Science, VNU-HCM

{23122044, 23122041, 23122039, 23122036, 23122048}@student.hcmus.edu.vn

87.2% Success on HG2.9K

FSE-AIWare 2026 · Montreal, Canada · July 5–9, 2026

Motivation

The Problem

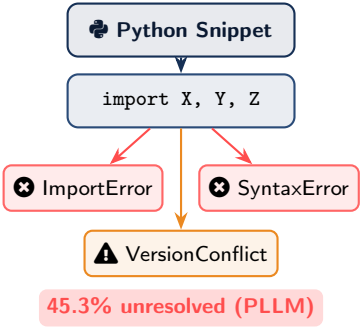
- Python has **500 000+** PyPI packages — dependency conflicts are *pervasive*.
- Legacy snippets frequently fail with `ImportError`, `SyntaxError`, version mismatch.

PLLM Baseline

- Only **54.7%** success rate on HG2.9K.
- 1 308 snippets remain *unresolved*.

Our Solution

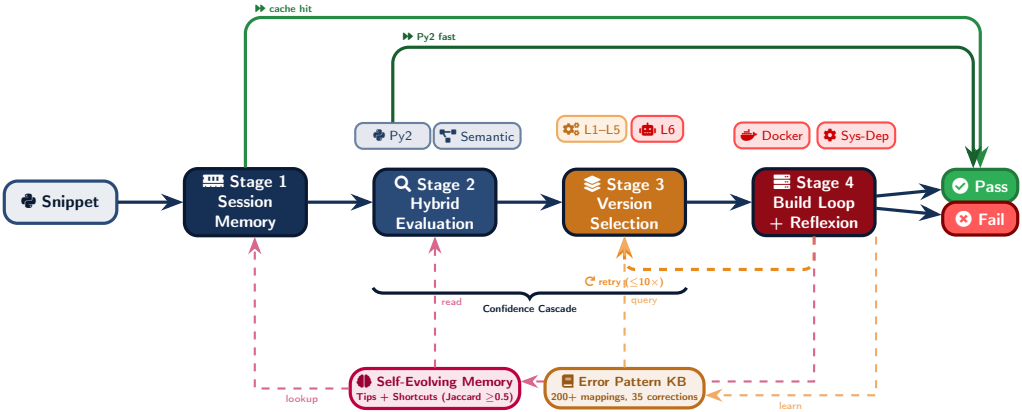
An **agentic, memory-augmented** approach that



FSE-AIWare 2026 Competition

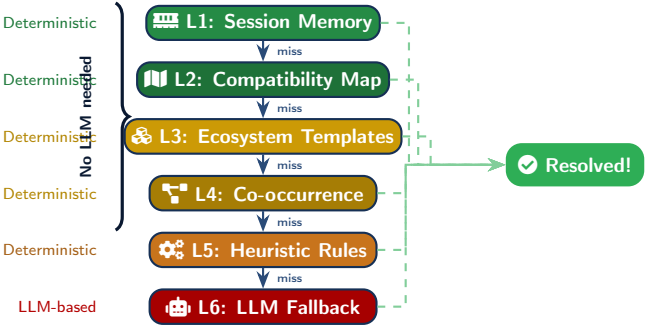
Montreal, Canada · July 5–9, 2026 · ACM Digital Library

MEMRES System Architecture



- Multi-level confidence cascade
- Memory-augmented: reuse proven solutions
- Error Pattern KB: 200+ mappings
- Py2 detector, sys-dep injection

Confidence Cascade



First hit terminates cascade · **60%+** fewer LLM calls

Key Techniques

Memory Systems

- **Intra-Session Memory**:
  Cache successful solutions for similar snippets.
- **Self-Evolving Memory**:
  Learn tips/shortcuts via Jaccard similarity.

Analysis

- **Semantic Analyzer**:
  Import usage patterns, ecosystem detection.
- **Python 2 Detector**:
  13 indicators, auto interpreter selection.

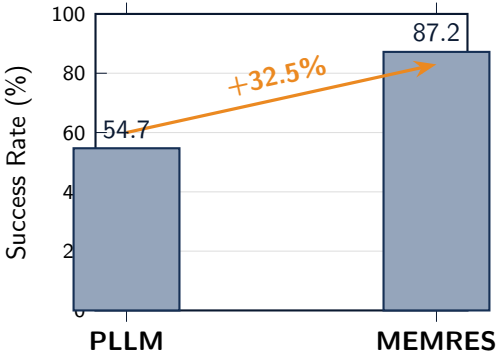
Knowledge Base

- **Error Pattern KB**:
  200+ mappings, name corrections, version constraints, regex rules.

Build & Deploy

- **System Dependency Injection**:
  `apt-get` for C-extensions, version fallback cascade.
- **Reflexion Loop**:
  Up to 10 retries with error-guided repair.

Evaluation: MEMRES vs. PLLM



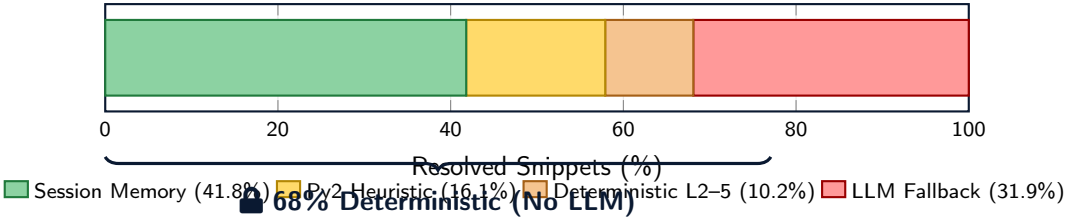
Metric	PLLM	MEMRES
Resolved	1 583	**2 519**
Failed	1 308	**371**
Success	54.7%	**87.2%**

Key Results

- Setup: HG2.9K, Gemma-2 9B, Docker
- **950+** PLLM failures resolved
- 53.9% needed sys-dep classifiers

Ablation Study: Resolution by Component

Total: 2 519 resolved (100%)



Memory

41.8% via cached solutions — zero LLM cost.

Python 2

Resolves the single largest PLLM failure category.

LLM

Only 32% need the LLM as last resort.

Failure Analysis (371 failures)

Category	Root Cause	Count	%
Build / C-ext	Missing C toolchains	186	50.1%
ImportError	Obscure/deprecated pkgs	121	32.6%
Timeout	Large ML compilation	55	14.8%
System/Platform	GTK, RPi, platform-only	9	2.4%
Total		**371**	**100%**

Inherent Limits

Most failures are **inherently unresolvable** in Docker — no C toolchain, no GPU.

Insight

32.6% from packages completely absent from PyPI (deprecated/renamed). Only 2.4% platform-specific.

Limitations & Lessons Learned

Limitations

- C-extension build errors dominate failures.
- Session memory is *order-sensitive*.
- Runtime passes (system-only imports) are strictly counted as failures.

Lessons

- Deterministic rules + memory outperform LLM-only.
- Error KB dramatically reduces repeat failures.
- Batch ordering affects memory effectiveness.

> *"The best LLM call is the one you never make."*
>
> **— Key design principle of MEMRES**

Conclusion

Summary

- MEMRES: agentic resolver with **memory augmentation** and **confidence cascade**.
- Achieves **87.2%** success on HG2.9K, surpassing PLLM (54.7%) by **+32.5%**.
- Reduces LLM calls by **60%+** via deterministic-first design.

Contributions

- Novel 4-stage pipeline with multi-level confidence cascade.
- Error Pattern KB (200+ mappings) + Self-Evolving Memory.
- Python 2 heuristics + System dependency injection.

2 519 / 2 890 snippets resolved (87.2%)

Thank You!

Questions & Discussion

87.2% Success Rate

{23122044, 23122041, 23122039, 23122030, 23122048}@student.hcmus.edu.vn

University of Science, VNU-HCM

github.com/checkdgt/fse-aiware-python-dependencies